

UNISENAI
Tecnologia em Análise e Desenvolvimento de Sistemas

Arthur Moreira
Douglas do Carmo
Jamil Cherem

SMARTRENT B2B
Plataforma de precificação preditiva de diárias para pequenos anfitriões e administradores
de imóveis da Grande Florianópolis

Florianópolis
2026

Arthur Moreira
Douglas do Carmo
Jamil Cherem

SMARTRENT B2B

Plataforma de precificação preditiva de diárias para pequenos anfitriões e administradores de imóveis da Grande Florianópolis

Relatório técnico apresentado à disciplina de Projeto Aplicado IV, do curso de Tecnologia em Análise e Desenvolvimento de Sistemas do UniSENAI, como parte da avaliação da unidade curricular.

Orientadora: Profa. Milena Maredmi Correa

Florianópolis
2026

RESUMO

Pequenos anfitriões e administradores de imóveis de temporada da Grande Florianópolis ainda controlam reservas por planilha ou aplicativo de mensagens e definem o preço da diária por intuição, sem levar em conta a sazonalidade ou a localização específica do imóvel. Esse cenário favorece a ocorrência de reservas sobrepostas para o mesmo período e mantém os imóveis ociosos fora da alta temporada. Este trabalho descreve o desenvolvimento do SmartRent B2B, uma plataforma para gestão de imóveis e reservas que inclui uma verificação automática de conflito de datas e um mecanismo de sugestão de preço baseado em inteligência artificial generativa, com um cálculo alternativo determinístico para os casos em que o serviço de IA não responde. O sistema foi construído em Java 17 e Spring Boot, com persistência em PostgreSQL, e alinha-se ao Objetivo de Desenvolvimento Sustentável 8 da Organização das Nações Unidas ao oferecer a um microempreendedor uma ferramenta de gestão antes restrita a operações de maior porte. São apresentados o levantamento de requisitos, a modelagem do domínio, os diagramas UML da solução e o estado atual da implementação.

Palavras-chave: SmartRent B2B; precificação dinâmica; gestão de reservas; sistemas distribuídos; Objetivo de Desenvolvimento Sustentável 8.

SUMÁRIO

1 INTRODUÇÃO	5
1.1 Contextualização e problema	5
1.2 Solução proposta	5
1.3 Alinhamento com o Objetivo de Desenvolvimento Sustentável 8	5
2 PERSONAS	7
2.1 Marina Andrade, pequena anfitriã	7
2.2 Carlos Bittencourt, administrador de imóveis	7
3 REQUISITOS	8
3.1 Requisitos funcionais	8
3.2 Requisitos não funcionais	11
4 CASOS DE USO	12
4.1 Gerenciar reservas e verificar conflito de datas	12
4.2 Gerar sugestão de preço com mecanismo de contingência	12
4.3 Demais casos de uso	13
5 DIAGRAMAS UML	14
5.1 Diagrama de classes	14
5.2 Diagrama de sequência da criação de reserva	14
5.3 Diagrama de sequência da sugestão de preço	15
6 MODELO DE DADOS	16
6.1 Modelo lógico	16
6.2 Modelo físico	16
7 TELAS	25
REFERÊNCIAS	26

1 INTRODUÇÃO

1.1 Contextualização e problema

A maior parte dos pequenos anfitriões e administradores de imóveis de temporada da Grande Florianópolis ainda depende de processos manuais para operar seus negócios: reservas anotadas em planilha ou combinadas por aplicativo de mensagens, e preço de diária definido por intuição ou por comparação informal com anúncios semelhantes. Não há, nesse processo, nenhuma ferramenta que relacione sazonalidade, localização específica do imóvel e perfil da propriedade para indicar um preço competitivo.

Esse modo de operação traz três consequências recorrentes. A primeira é o *overbooking*: sem uma verificação automática, é possível aceitar duas reservas para o mesmo imóvel em datas que se sobrepõem. A segunda é a ociosidade dos imóveis fora da alta temporada, decorrente da ausência de uma estratégia de preço que reaja à demanda ao longo do ano. A terceira é a exclusão do pequeno empreendedor das ferramentas de revenue management hoje disponíveis no mercado, que costumam ser corporativas, caras e voltadas a redes hoteleiras de grande porte, não a quem administra um punhado de imóveis.

1.2 Solução proposta

O SmartRent B2B é uma plataforma para gestão de imóveis e reservas de temporada, desenvolvida com escopo delimitado por uma matriz MoSCoW para caber em um ciclo de 30 dias. O sistema mantém um cadastro de imóveis e reservas com verificação automática de conflito de datas, calcula uma sugestão de preço de diária combinando sazonalidade, microgeografia (latitude e longitude do imóvel) e perfil da propriedade por meio de inteligência artificial generativa, e recorre a uma regra de negócio determinística sempre que essa IA está indisponível, registrando qual dos dois caminhos gerou cada sugestão. Um *dashboard* reúne indicadores de ocupação e de evolução de preço para o anfitrião. A autenticação de usuários é reaproveitada do Projeto Aplicado III, evitando retrabalho em um domínio já resolvido pela equipe em semestre anterior.

1.3 Alinhamento com o Objetivo de Desenvolvimento Sustentável 8

O projeto se relaciona com o Objetivo de Desenvolvimento Sustentável 8 da Agenda 2030, voltado ao trabalho decente e ao crescimento econômico (ORGANIZAÇÃO DAS NAÇÕES UNIDAS, 2015), em três de suas metas.

Meta	Contribuição do SmartRent B2B
8.2 — diversificação, modernização tecnológica e inovação produtiva	Leva a um microempreendedor de temporada um instrumento de precificação orientado a dados, hoje restrito a redes hoteleiras de maior porte.
8.3 — apoio às micro e pequenas empresas e ao empreendedorismo	Substitui o controle informal por planilha e mensagens por um registro único de reservas e indicadores, aproximando a operação do anfitrião de uma gestão profissional.
8.9 — turismo sustentável, geração de emprego e valorização da cultura local	A sugestão de preço sensível à sazonalidade favorece uma distribuição mais equilibrada da ocupação ao longo do ano, reduzindo a ociosidade típica da baixa temporada no litoral catarinense.

Fonte: elaborado pelos autores (2026).

2 PERSONAS

As personas a seguir sintetizam os dois perfis de usuário considerados no levantamento de requisitos do MVP.

2.1 Marina Andrade, pequena anfitriã

Marina tem 34 anos, é professora de yoga e administra três imóveis de temporada próprios, em Canasvieiras e em Jurerê. Ela controla as reservas por uma planilha compartilhada e por conversas de WhatsApp com os hóspedes, e já enfrentou um caso de *overbooking*: aceitou duas reservas para o mesmo apartamento no mesmo fim de semana. O preço da diária é definido observando anúncios parecidos em outras plataformas, sem um critério mais sistemático. Marina não tem tempo nem conhecimento técnico para operar uma ferramenta de revenue management corporativa; do SmartRent B2B ela espera sobretudo a garantia de que duas reservas nunca vão colidir e uma sugestão de preço simples, que possa aceitar ou ajustar sem precisar entender a lógica estatística por trás do número.

2.2 Carlos Bittencourt, administrador de imóveis

Carlos tem 45 anos, é ex-corretor de imóveis e administra uma carteira de catorze imóveis de terceiros na Grande Florianópolis, repassando parte da receita aos proprietários. Hoje mantém três planilhas separadas, para reservas, repasses e manutenção, que com frequência saem de sincronia entre si. Sua precificação é reativa: só reduz o valor da diária quando percebe queda na procura, geralmente tarde demais para recuperar a ocupação do período. Carlos já avaliou contratar um PMS (*property management system*) internacional, mas considerou as opções do mercado caras e complexas para o porte da sua operação. Do sistema, ele espera um painel único com visão consolidada de todos os imóveis e uma justificativa auditável para cada preço sugerido, que possa apresentar aos proprietários.

Uma persona secundária, beneficiada de forma indireta, é o próprio hóspede: ele se beneficia de um preço mais coerente com a demanda e de menor risco de encontrar sua reserva cancelada por conflito de agenda, ainda que não interaja diretamente com o sistema no escopo deste MVP.

3 REQUISITOS

Os requisitos foram organizados segundo a matriz MoSCoW (Must, Should, Could, Won't) definida para o MVP de 30 dias; itens classificados como Won't foram registrados como Visão Futura e não fazem parte do escopo atual.

3.1 Requisitos funcionais

Quadro 2 – Requisitos funcionais do MVP

ID	Descrição	MoSCoW
RF01	O sistema deve permitir que o anfitrião se autentique com as credenciais e a infraestrutura de autenticação herdadas do Projeto Aplicado III.	Must
RF02	O sistema deve permitir cadastrar um imóvel com título, tipo, endereço completo (incluindo latitude e longitude), capacidade, número de quartos e banheiros, valor de diária base e comodidades.	Must
RF03	O sistema deve permitir editar os dados de um imóvel e inativá-lo.	Must
RF04	O sistema deve permitir listar os imóveis de um anfitrião, com filtro por situação ativa ou inativa.	Must
RF05	O sistema deve permitir cadastrar uma reserva vinculada a um imóvel, com os dados do hóspede e o período de entrada e saída.	Must
RF06	O sistema deve impedir o cadastro ou a edição de uma reserva cujo período se sobreponha a outra reserva ativa do mesmo imóvel.	Must
RF07	O sistema deve permitir cancelar ou concluir uma reserva, atualizando sua situação.	Must
RF08	O sistema deve calcular uma sugestão de preço de diária para um imóvel em uma data de referência, considerando sazonalidade e microgeografia, por meio de inteligência artificial generativa.	Must
RF09	O sistema deve aplicar uma regra de negócio determinística quando a IA generativa estiver indisponível ou o tempo de resposta for excedido, registrando a origem de cada cálculo.	Must
RF10	O sistema deve exibir um painel com indicadores de ocupação e evolução dos preços sugeridos por imóvel.	Should
RF11	O sistema deve permitir consultar o histórico de sugestões de preço de um imóvel em um intervalo de datas.	Should
RF12	Ao recusar uma reserva por conflito de datas, o sistema deve indicar quais reservas existentes causaram o conflito.	Should
RF13	Sincronização bidirecional de reservas com plataformas externas via iCal.	Won't (Visão Futura)
RF14	Processamento de pagamentos reais por meio de um gateway.	Won't (Visão Futura)

ID	Descrição	MoSCoW
RF15	O sistema deve permitir ao gestor cadastrar um anúncio com título, descrição, características, endereço completo, até 14 imagens (fotos comuns e em 360 graus somadas), até 2 vídeos de no máximo 1 minuto e 30 segundos, quantidade mínima de diárias, taxa de limpeza, limite de hóspedes e aceite obrigatório do termo de uso, validando limites e formatos também no servidor. O contato com o gestor é exclusivamente pelo SmartChat: não há campo nem link de WhatsApp.	Must
RF16	O anúncio deve seguir um ciclo de vida com transições validadas no servidor: pré-publicação sem preço, janela de 24 horas contada da primeira confirmação de preço (que não reinicia quando o preço muda), pronto para publicar e publicado, este último exigindo novo aceite do termo.	Must
RF17	O sistema deve validar no servidor que o usuário comum acessa apenas o catálogo, que o gestor vê e altera somente os próprios imóveis, reservas, bloqueios, estatísticas e conversas.	Must
RF18	O sistema deve permitir editar um anúncio publicado: ele sai do catálogo ao iniciar a edição, o progresso fica em rascunho, a confirmação o republica automaticamente após 2 horas e o descarte o devolve ao ar imediatamente, sem prazo máximo de edição.	Must
RF19	O sistema deve notificar o gestor, sem alterar o anúncio, quando uma edição completar 24 horas aberta, repetindo o lembrete em intervalo configurável.	Should
RF20	Cada reserva deve gravar um registro imutável do preço e dos dados do imóvel no momento da criação, de modo que alterações posteriores de preço ou de anúncio não afetem reservas existentes.	Must
RF21	O sistema deve permitir solicitar sugestão de preço por IA para vários imóveis em pré-publicação, com revisão e confirmação explícitas do gestor e histórico de origem (manual ou IA) de cada alteração de preço.	Should
RF23	O Dashboard deve exibir somente estatísticas dos imóveis do gestor autenticado (reservas por mês, ocupação, receita líquida de reembolsos, imóveis por situação), sem ações operacionais; cancelamentos não contam como ocupação.	Should
RF24	O Pannel do Gestor deve exibir um calendário de três meses por imóvel selecionado, com reservas confirmadas em cinza, pendentes e bloqueios identificados também sem depender de cor, e detalhes do hóspede visíveis apenas ao gestor dono.	Must
RF25	O gestor deve poder bloquear e desbloquear datas do imóvel (total ou parcialmente), sem conflito com reservas confirmadas, de forma segura sob concorrência, com auditoria; para clientes as datas aparecem apenas como indisponíveis.	Must
RF26	A reserva deve registrar de forma imutável o mínimo de diárias, a taxa de limpeza, o limite de hóspedes e os parâmetros da política de cancelamento vigentes; o servidor rejeita reservas fora do mínimo ou do limite.	Must
RF27	O cliente logado deve poder reservar e pagar (gateway simulado, com chave de idempotência), sabendo antes do pagamento a política e se a reserva nasce sem direito a reembolso.	Must

ID	Descrição	MoSCoW
RF28	O cancelamento deve seguir a regra decidida no servidor: reembolso integral até 2 dias antes do horário de check-in, sem reembolso depois, reembolso integral quando o gestor cancela e cancelamento sem cobrança para reservas pendentes, com reembolso idempotente, retry em falha do gateway e liberação imediata das datas.	Must
RF29	O sistema deve oferecer o SmartChat como canal único entre cliente e gestor, com conversa única por cliente, gestor e imóvel, criada automaticamente de forma idempotente na confirmação da reserva, tempo quase real e contador de não lidas.	Must
RF30	As mensagens do SmartChat devem ter telefones, links externos, e-mails e conteúdo impróprio ocultados no servidor, preservando o original apenas em campo restrito; denúncia e solicitação de bloqueio de usuário são registradas (protótipo, sem efeito funcional).	Must
RF31	Vídeos do anúncio têm no máximo 1:30, são enviados em partes retomáveis, validados no servidor e processados de forma assíncrona (estados ENVIANDO, PROCESSANDO, PRONTO e FALHA); só vídeos prontos aparecem e a publicação os exige.	Must
RF32	Todas as regras e exhibições que dependem de dia ou hora seguem o horário de Brasília (America/Sao_Paulo), com instantes novos em UTC, datas de calendário sem fuso e prazos como tempo decorrido.	Must
RF33	A região do usuário no cabeçalho (avatar e nome) deve levar à página de perfil, acessível por teclado, mostrando a foto quando existir e, caso contrário, as iniciais; a página exige login e cada usuário vê e altera apenas o próprio perfil.	Must
RF34	O usuário deve poder alterar o nome de exibição (2 a 60 caracteres, sem HTML, telefone, e-mail, link ou termos ofensivos), sem alterar reservas, snapshots ou auditorias já gravados.	Must
RF35	O usuário deve poder enviar e remover a foto de perfil: apenas PNG ou JPG, tipo verificado pelo conteúdo, limites de tamanho e dimensões, reprocessada no servidor (recorte quadrado, 512 por 512, sem metadados), uma por usuário.	Must
RF36	O usuário deve poder alterar a senha informando a atual, com política da nova senha (tamanho mínimo, lista local de senhas comuns, diferente da atual), encerramento das demais sessões e aviso por e-mail.	Must
RF37	A troca do e-mail de login deve exigir a senha atual e a confirmação por link de uso único e validade curta enviado ao novo endereço, com resposta neutra, aviso ao endereço antigo e encerramento das demais sessões.	Must
RF38	O usuário deve poder solicitar a exclusão de dados e cancelá-la; nada é excluído automaticamente: o pedido é analisado pela equipe, com período mínimo de análise, e-mail de confirmação com link para cancelar e sinais de risco registrados.	Must
RF39	Enquanto houver solicitação de exclusão em andamento, devem valer restrições temporárias parciais (novas reservas, publicação e confirmação de edição, e-mail), sem bloquear login, senha, chat, reservas confirmadas, reembolsos ou bloqueio de datas; a aprovação é barrada por reservas futuras, reembolso pendente ou denúncia aberta.	Must
RF22	O sistema deve manter trilha de auditoria dos aceites do termo (usuário, imóvel, versão, data, hora e IP), das alterações de preço e das ações sobre o anúncio.	Should

Fonte: elaborado pelos autores (2026).

3.2 Requisitos não funcionais

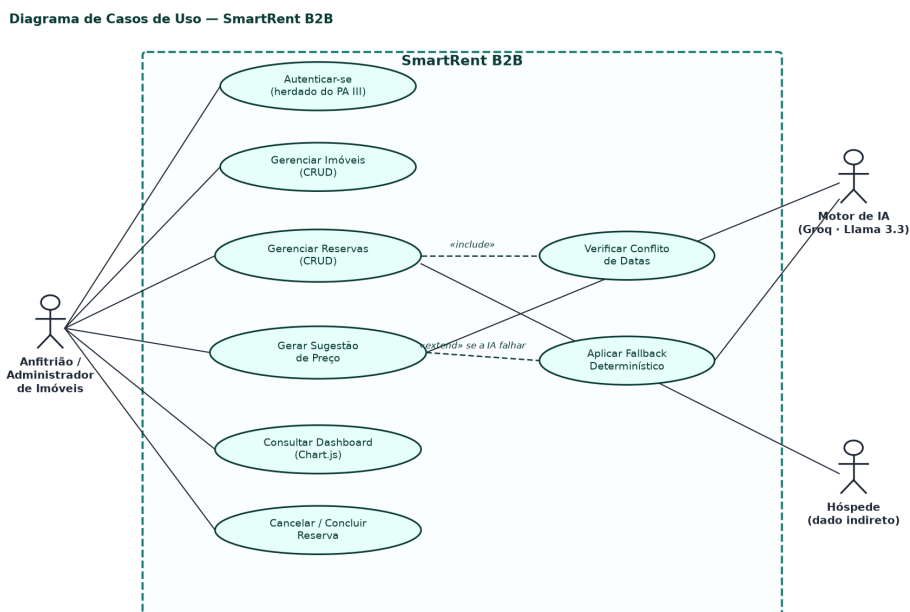
Quadro 3 – Requisitos não funcionais do MVP

ID	Descrição	Categoria
RNF01	O backend deve ser implementado em Java 17 com Spring Boot 3, usando Spring Web, Spring Data JPA e o cliente HTTP nativo (RestClient).	Tecnologia
RNF02	Os dados devem ser persistidos em PostgreSQL, gerenciado por meio do Supabase.	Tecnologia
RNF03	Toda credencial ou chave de API deve ser fornecida ao sistema por variável de ambiente, nunca escrita diretamente no código-fonte.	Segurança
RNF04	As chamadas à IA generativa e a quaisquer serviços externos devem ser simuladas (mock) nos testes automatizados, de modo que o build não dependa de rede.	Testabilidade
RNF05	O esquema do banco de dados deve ser versionado por migration; em produção, o Hibernate deve apenas validar o esquema existente, nunca alterá-lo automaticamente.	Manutenibilidade
RNF06	A interface deve ser responsiva, construída com HTML5 semântico, JavaScript assíncrono e Tailwind CSS, com prioridade ao layout para dispositivos móveis.	Usabilidade
RNF07	As operações de cadastro, consulta, alteração e remoção devem responder em tempo compatível com uso interativo, excluindo a latência da chamada externa à IA.	Desempenho
RNF08	O código deve seguir os princípios SOLID e a separação em camadas de controlador, serviço e repositório, com objetos de transferência de dados implementados como Records.	Manutenibilidade
RNF09	A esteira de integração contínua deve executar a compilação e os testes a cada envio de código, impedindo a integração de alterações que quebrem o build.	Confiabilidade
RNF10	Senhas, hashes e tokens nunca devem ser registrados em log nem na auditoria; tokens de uso único são guardados somente como hash.	Segurança
RNF11	Ações sensíveis da conta (senha, e-mail, exclusão de dados) devem exigir reautenticação, limitar tentativas incorretas e ser auditadas com usuário, endereço IP e instante em UTC.	Segurança
RNF12	O texto das mensagens do SmartChat, o das denúncias e a prévia das notificações devem ser gravados cifrados em repouso (AES-256-GCM, chave fora do código) e o texto das mensagens não pode ser registrado em log; o servidor continua lendo o texto para filtro e denúncia.	Segurança

Fonte: elaborado pelos autores (2026).

4 CASOS DE USO

Figura 1 – diagrama de casos de uso do SmartRent B2B



Fonte: elaborado pelos autores (2026).

4.1 Gerenciar reservas e verificar conflito de datas

Quadro 4 – Caso de uso — gerenciar reservas e verificar conflito de datas

Ator principal	Anfitrião ou administrador de imóveis
Pré-condição	Anfitrião autenticado; imóvel já cadastrado e ativo
Fluxo principal	O anfitrião informa os dados da reserva (imóvel, hóspede, datas de entrada e saída, valor). O sistema verifica se há sobreposição de datas para o mesmo imóvel, desconsiderando reservas já canceladas. Não havendo conflito, a reserva é registrada com situação pendente e o sistema confirma a operação.
Fluxo alternativo	Havendo conflito, o sistema recusa a operação e retorna as reservas que causaram a sobreposição, sem registrar nada.
Pós-condição	A reserva passa a constar na listagem do imóvel e nenhuma sobreposição de datas ativa existe para ele.

Fonte: elaborado pelos autores (2026).

4.2 Gerar sugestão de preço com mecanismo de contingência

Quadro 5 – Caso de uso — gerar sugestão de preço com mecanismo de contingência

Ator principal	Anfitrião ou administrador de imóveis
Ator secundário	Serviço de IA generativa (Grok / Llama 3.3)
Pré-condição	Imóvel cadastrado com endereço geocodificado
Fluxo principal	O anfitrião solicita a sugestão de preço para um imóvel e uma data de referência. O sistema monta o contexto de sazonalidade e microgeografia e consulta a IA generativa, que devolve o valor sugerido, os fatores de ajuste considerados e uma justificativa textual. A sugestão é registrada com a origem marcada como IA generativa.
Fluxo alternativo	Se a chamada à IA falhar ou exceder o tempo limite, o sistema aplica uma regra determinística de contingência e registra a sugestão com a origem marcada como cálculo de contingência, de modo que o anfitrião nunca fique sem um valor sugerido.
Pós-condição	A sugestão fica disponível para consulta e exibição no painel, com a origem do cálculo identificada.

Fonte: elaborado pelos autores (2026).

4.3 Demais casos de uso

Quadro 6 – Demais casos de uso do MVP

Caso de uso	Resumo
Autenticar-se	Login com e-mail e senha, reaproveitando a infraestrutura de segurança do Projeto Aplicado III.
Gerenciar imóveis	Cadastro, edição, listagem e inativação de imóveis.
Consultar painel	Visualização de indicadores de ocupação e evolução de preço sugerido.
Cancelar ou concluir reserva	Atualização da situação de uma reserva; reservas canceladas liberam o período para uma nova reserva.
Criar e publicar anúncio	Cadastro com mídias e termo de uso, definição de preço (manual ou por IA), janela de 24 horas e publicação com novo aceite (RF15, RF16, RF21).
Editar anúncio publicado	Início da edição (anúncio sai do ar), rascunho, confirmação com republicação em 2 horas ou descarte imediato (RF18).
Gerenciar o próprio perfil	Acesso pelo cabeçalho, alteração do nome de exibição e envio ou remoção da foto de perfil (RF33 a RF35).
Alterar senha e e-mail	Reautenticação, política da nova senha, encerramento das demais sessões e troca de e-mail confirmada por link de uso único (RF36, RF37).
Solicitar exclusão de dados	Pedido com senha atual, e-mail com link para cancelar, restrições temporárias durante a análise e cancelamento a qualquer momento (RF38, RF39).
Analisar solicitação de exclusão	Aprovação, negativa ou conclusão pela equipe, com período mínimo e trava por impedimentos; serviço pronto, sem tela (depende do módulo de administração).

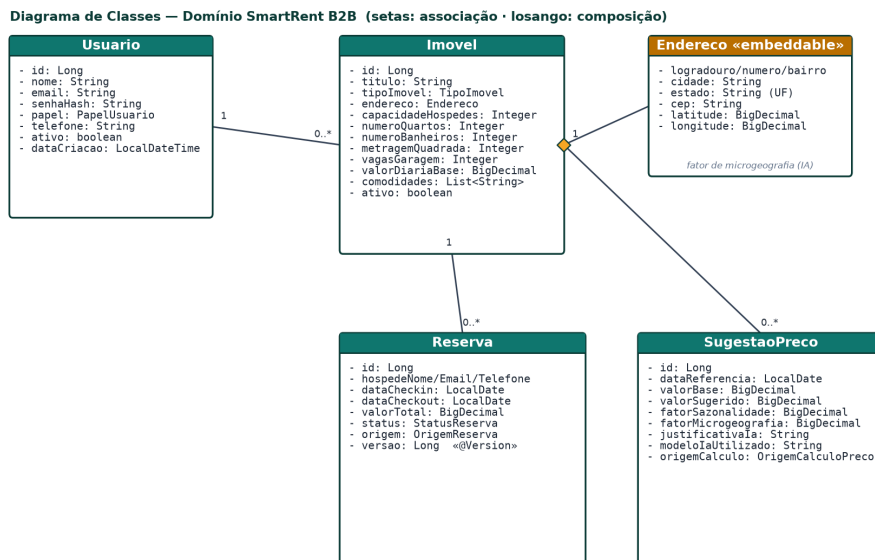
Fonte: elaborado pelos autores (2026).

5 DIAGRAMAS UML

5.1 Diagrama de classes

O diagrama a seguir representa as entidades do domínio e seus relacionamentos: um usuário possui vários imóveis; cada imóvel possui várias reservas e várias sugestões de preço ao longo do tempo. O endereço é modelado como um objeto de valor incorporado ao imóvel, pois é nele que residem a latitude e a longitude usadas pelo cálculo de microgeografia.

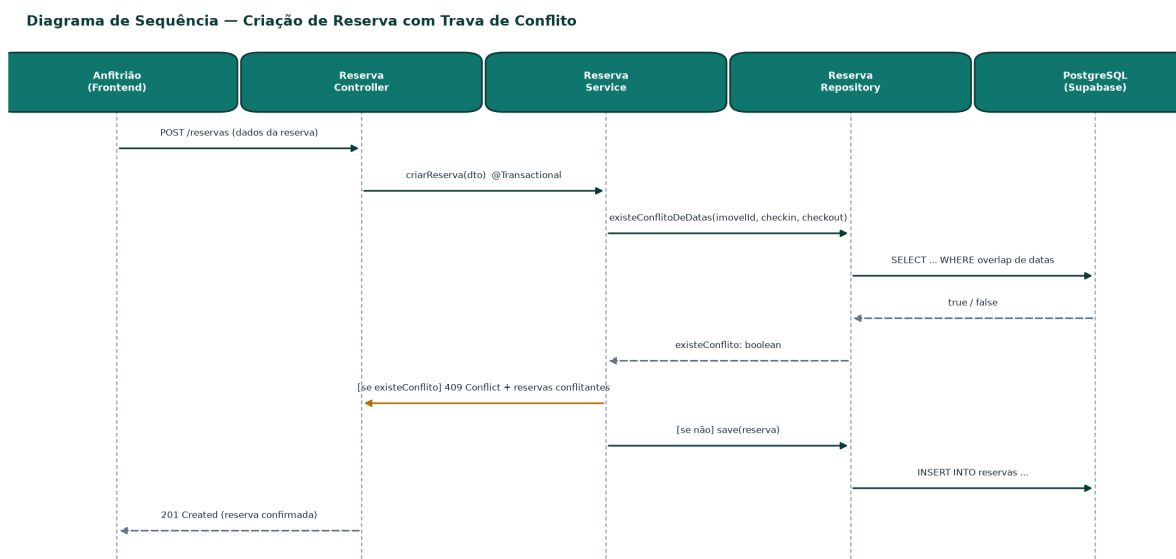
Figura 2 – diagrama de classes do domínio



Fonte: elaborado pelos autores (2026).

5.2 Diagrama de sequência da criação de reserva

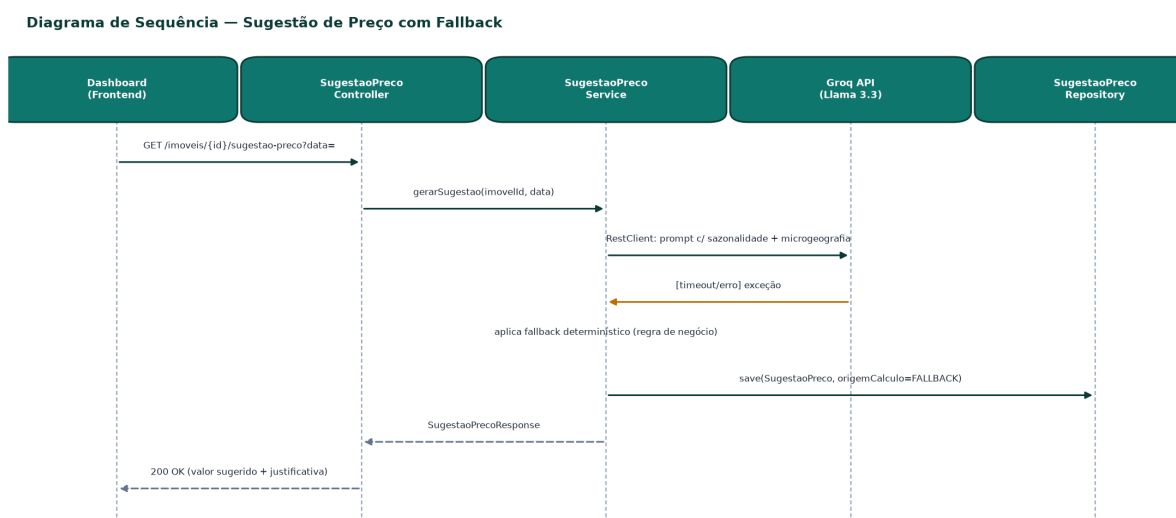
Figura 3 – criação de reserva com verificação de conflito de datas



Fonte: elaborado pelos autores (2026).

5.3 Diagrama de sequência da sugestão de preço

Figura 4 – geração da sugestão de preço com desvio para o cálculo de contingência



Fonte: elaborado pelos autores (2026).

6 MODELO DE DADOS

6.1 Modelo lógico

O modelo lógico corresponde ao diagrama de classes apresentado na Seção 5.1: quatro entidades principais (usuário, imóvel, reserva e sugestão de preço) e um objeto de valor incorporado (endereço). Todos os relacionamentos entre as entidades principais são de um para muitos. O fluxo de anúncios acrescentou entidades de apoio (mídias, rascunho de edição, histórico de preço, aceites do termo, auditoria, lembretes e notificações), descritas na Seção 6.2 e ainda não representadas no diagrama da Seção 5.1.

6.2 Modelo físico

O esquema físico, em PostgreSQL, está especificado nos quadros a seguir e encontra-se implementado. Ele foi gerado a partir do mapeamento objeto-relacional em modo somente-geração-de-script do Hibernate (FIELDING; BAUER; KING, [2024] apud documentação do projeto) e é versionado por migrations aplicadas pelo Flyway na inicialização da aplicação, cabendo ao Hibernate apenas validar o esquema existente, nunca alterá-lo. A conformidade entre o mapeamento e os quadros a seguir foi verificada contra uma instância PostgreSQL 16 em ambiente de desenvolvimento.

Quadro 7 – Estrutura da tabela usuarios

Coluna	Tipo	Observação
id	bigserial	chave primária
nome	varchar(120)	obrigatório
email	varchar(150)	obrigatório, único
senha_hash	varchar(60)	obrigatório
papel	varchar(20)	obrigatório
telefone	varchar(20)	opcional
ativo	boolean	obrigatório
data_criacao	timestamp	obrigatório
smartchat_liberado	boolean	aba SmartChat do cliente; guardada no backend
foto_arquivo, foto-versao	varchar(64) / integer	foto de perfil: nome aleatório e versão contra cache
sessao-versao	integer	sobe na troca de senha ou e-mail; o token só vale na versão atual
senha_definida	boolean	falso em conta criada pelo Google, sem senha conhecida

Fonte: elaborado pelos autores (2026).

Quadro 8 – Estrutura da tabela imoveis

Coluna	Tipo	Observação
id	bigserial	chave primária
usuario_id	bigint	obrigatório, referencia usuarios
titulo	varchar(150)	obrigatório
descricao	text	opcional
tipo_imovel	varchar(20)	obrigatório
endereço (logradouro, bairro, cidade, estado, cep)	varchar	obrigatório
latitude, longitude	numeric(10,7)	opcional — microgeografia
capacidade_hospedes, numero_quartos, numero_banheiros	integer	obrigatório
metragem_quadrada	integer	opcional — área útil em m ²
vagas_garagem	integer	opcional — nulo e zero são distintos
valor_diaria_base	numeric(10,2)	opcional até a confirmação do preço
status	varchar(30)	obrigatório — ciclo de vida do anúncio
preco_primeira_confirmacao_em	timestamp	âncora da janela de 24h; gravado uma única vez
publicado_em, edicao_iniciada_em, edicao_estado_origem, republicar_original_em, edicao_confirmada_em, republicar_em	timestamp / varchar	fases de edição e republicação
minimo_diarias, taxa_limpeza	integer / numeric(10,2)	mínimo de diárias (padrão 1) e taxa de limpeza por reserva (padrão 0)
ativo	boolean	obrigatório
data_cadastro	timestamp	obrigatório

Fonte: elaborado pelos autores (2026).

Quadro 9 – Estrutura da tabela reservas

Coluna	Tipo	Observação
id	bigserial	chave primária
imovel_id	bigint	obrigatório, referencia imoveis
hospede_nome, hospede_email	varchar	obrigatório
cliente_id, numero_hospedes	bigint / integer	cliente dono da reserva (opcional) e quantidade de hóspedes
data_checkin, data_checkout	date	obrigatório
valor_total	numeric(10,2)	obrigatório
status, origem	varchar(20)	obrigatório

Coluna	Tipo	Observação
observacoes	text	opcional
data_criacao	timestamp	obrigatório
versao	bigint	controle de concorrência otimista
minimo_diarias_snapshot, taxa_limpeza_snapshot, limite_hospedes_snapshot	integer / numeric	termos do anúncio vigentes na criação (imutáveis)
politica_antecedencia_horas, politica_regret_dias, politica_versao	integer / varchar	política de cancelamento vigente na criação (imutável)
cancelada_em, cancelada_por, cancelamento_regra, cancelamento_motivo	timestampz / varchar	registro do cancelamento; status inclui CANCELADA_COM_REEMBOLSO, CANCELADA_SEM_REEMBOLSO e CANCELADA_PELLO_GESTOR
moeda, numero_diarias, preco_diaria_snapshot, taxas_snapshot, total_snapshot	varchar / integer / numeric	snapshot imutável das condições financeiras
imovel_titulo_snapshot, imovel_endereco_snapshot, imovel_caracteristicas_snapshot	varchar	snapshot dos dados do imóvel na criação

Fonte: elaborado pelos autores (2026).

Quadro 10 – Estrutura da tabela sugestoes_preco

Coluna	Tipo	Observação
id	bigserial	chave primária
imovel_id	bigint	obrigatório, referencia imoveis
data_referencia	date	obrigatório
valor_base, valor_sugerido	numeric(10,2)	obrigatório
percentual_ajuste, fator_sazonalidade, fator_microgeografia	numeric(5,2)	opcional
justificativa_ia	text	opcional
modelo_ia_utilizado	varchar(80)	opcional
origem_calculo	varchar(30)	obrigatório — IA generativa ou contingência
data_geracao	timestamp	obrigatório

Fonte: elaborado pelos autores (2026).

Quadro 11 – Estrutura da tabela reserva_precos_diarios

Coluna	Tipo	Observação
reserva_id, data	bigint, date	chave composta — valor de cada diária da reserva
valor	numeric(10,2)	obrigatório — gravado uma única vez

Fonte: elaborado pelos autores (2026).

Quadro 12 – Estrutura da tabela imovel_midias

Coluna	Tipo	Observação
id	bigserial	chave primária
imovel_id	bigint	obrigatório, referencia imoveis
chave	varchar(36)	obrigatório, único — UUID da URL pública
tipo	varchar(10)	FOTO, FOTO_360 ou VIDEO
estado	varchar(10)	ATIVA, NOVA ou REMOVIDA (edição em andamento)
arquivo, miniatura, mime, tamanho_bytes	varchar / bigint	metadados do arquivo guardado
largura, altura, duracao_segundos	integer	opcionais — lidos no servidor
ordem, capa, data_envio	integer / boolean / timestamp	obrigatório
status_processamento, motivo_falha	varchar	vídeo: ENVIANDO, PROCESSANDO, PRONTO ou FALHA, com motivo
poster, hls_mestre, original_arquivo	varchar(160)	arquivos gerados no processamento do vídeo
tentativas, proxima_tentativa_em	integer / timestamptz	fila e retry com espera crescente
tamanho_total, bytes_recebidos, atualizado_em	bigint / timestamptz	envio em partes retomável e limpeza de órfãos

Fonte: elaborado pelos autores (2026).

Quadro 13 – Estrutura da tabela anuncio_rascunhos

Coluna	Tipo	Observação
id	bigserial	chave primária
imovel_id	bigint	obrigatório, único
dados	text	campos do formulário em JSON
midia_ordem, capa_midia_id	text / bigint	ordem e capa escolhidas na edição
criado_em, atualizado_em	timestamp	obrigatório

Fonte: elaborado pelos autores (2026).

Quadro 14 – Estrutura da tabela historico_preco

Coluna	Tipo	Observação
id	bigserial	chave primária
imovel_id, autor_id	bigint	obrigatório

Coluna	Tipo	Observação
valor_anterior, valor_novo	numeric(10,2)	valor_anterior é nulo na primeira definição
origem	varchar(10)	MANUAL ou IA
data_hora	timestamp	obrigatório

Fonte: elaborado pelos autores (2026).

Quadro 15 – Estrutura da tabela aceites_termo

Coluna	Tipo	Observação
id	bigserial	chave primária
usuario_id, imovel_id	bigint	obrigatório
versao_termo	varchar(20)	obrigatório
contexto	varchar(30)	CADASTRO, PUBLICACAO ou CONFIRMACAO_EDICAO
data_hora, ip	timestamp / varchar(45)	auditoria do aceite

Fonte: elaborado pelos autores (2026).

Quadro 16 – Estrutura da tabela auditoria_anuncio

Coluna	Tipo	Observação
id	bigserial	chave primária
imovel_id	bigint	obrigatório
usuario_id	bigint	nulo quando a ação parte do sistema
acao, detalhes, data_hora	varchar / timestamp	trilha das ações sobre o anúncio

Fonte: elaborado pelos autores (2026).

Quadro 17 – Estrutura da tabela lembretes_edicao

Coluna	Tipo	Observação
id	bigserial	chave primária
imovel_id	bigint	obrigatório
edicao_iniciada_em, numero, canal	timestamp / integer / varchar	chave única — garante o envio sem duplicidade
criado_em, enviado_em, tentativas, erro	timestamp / integer / varchar	enviado_em nulo indica entrega pendente de nova tentativa

Fonte: elaborado pelos autores (2026).

Quadro 18 – Estrutura da tabela pagamentos

Coluna	Tipo	Observação
id, reserva_id	bigserial / bigint	cobrança de uma reserva

Coluna	Tipo	Observação
valor, moeda, status	numeric / varchar	APROVADO ou RECUSADO
chave_idempotencia	varchar(100)	única — impede cobrança dupla
ref_gateway, motivo_recusa, criado_em	varchar / timestampz	referência do gateway e instante (UTC)

Fonte: elaborado pelos autores (2026).

Quadro 19 – Estrutura da tabela reembolsos

Coluna	Tipo	Observação
id, reserva_id	bigserial / bigint	um estorno por reserva (único)
valor, status	numeric / varchar	PENDENTE, PROCESSADO ou FALHA
chave_idempotencia	varchar(100)	única — impede estorno duplicado
tentativas, erro, proxima_tentativa_em	integer / varchar / timestampz	retry com espera crescente e alerta

Fonte: elaborado pelos autores (2026).

Quadro 20 – Estrutura da tabela bloqueios_datas

Coluna	Tipo	Observação
id, imovel_id	bigserial / bigint	bloqueio manual de datas de um imóvel
data_inicio, data_fim	date	datas puras, inclusivas, sem fuso
motivo, observacao	varchar	internos: nunca expostos a clientes
criado_por, criado_em	bigint / timestampz	auditoria

Fonte: elaborado pelos autores (2026).

Quadro 21 – Estrutura da tabela smartchat_conversas

Coluna	Tipo	Observação
id, cliente_id, gestor_id, imovel_id	bigserial / bigint	restrição de unicidade (cliente, gestor, imóvel)
criada_em, ultima_mensagem_em	timestampz	ordenação da lista

Fonte: elaborado pelos autores (2026).

Quadro 22 – Estrutura da tabela smartchat_mensagens

Coluna	Tipo	Observação
id, conversa_id, autor_id	bigserial / bigint	autor nulo nas mensagens de sistema
tipo, texto_filtrado	varchar	NORMAL ou SISTEMA; único texto que sai do servidor

Coluna	Tipo	Observação
texto_original, categorias, ocorrencias	varchar / integer	restritos ao backend (moderação futura e métricas)
chave_idempotencia, criada_em, lida_em	varchar / timestamptz	chave única nas mensagens de sistema

Fonte: elaborado pelos autores (2026).

Quadro 23 – Estrutura da tabela smartchat_denuncias e smartchat_bloqueios_usuario

Coluna	Tipo	Observação
conversa_id, denunciante_id, denunciado_id	bigint	denúncia registrada (status PENDENTE)
motivo, descricao, mensagens_anexadas	varchar	dados para o futuro módulo de administração
bloqueador_id, bloqueado_id, status	bigint / varchar	pedido de bloqueio, sem efeito funcional

Fonte: elaborado pelos autores (2026).

Quadro 24 – Estrutura da tabela auditoria_conta

Coluna	Tipo	Observação
id, usuario_id	bigserial / bigint	trilha de eventos da conta
acao, detalhes	varchar	sem valores sensíveis: nunca senha, hash ou token
ip, ocorrida_em	varchar(45) / timestamptz	endereço IP e instante em UTC

Fonte: elaborado pelos autores (2026).

Quadro 25 – Estrutura da tabela troca_email

Coluna	Tipo	Observação
id, usuario_id	bigserial / bigint	no máximo um pedido pendente por usuário (índice único parcial)
email_novo	varchar(150)	só passa a ser o e-mail da conta ao confirmar o link
token_hash	varchar(64)	único — somente o SHA-256 do token; o token só vai no link
criada_em, expira_em, usado_em, cancelada_em	timestamptz	validade curta e uso único

Fonte: elaborado pelos autores (2026).

Quadro 26 – Estrutura da tabela solicitacoes_exclusao

Coluna	Tipo	Observação
id, usuario_id	bigserial / bigint	uma solicitação em andamento por usuário (índice único parcial)

Coluna	Tipo	Observação
estado	varchar(30)	PENDENTE, EM_ANALISE, APROVADA, NEGADA, CONCLUIDA ou CANCELADA_PELo_USUARIO
email_no_pedido, motivo, ip	varchar	cópia do e-mail no pedido e motivo opcional do usuário
sinais_risco	varchar(2000)	sim/não e contagens para a equipe; nenhum dado pessoal
criada_em, analise_apos	timestampz	a equipe só pode aprovar depois do período mínimo
analista_id, decidida_em, observacao_interna	bigint / timestampz / varchar	nulos até o módulo de administração
token_cancelamento_hash	varchar(64)	único — link “não fui eu”, somente o hash

Fonte: elaborado pelos autores (2026).

Quadro 27 – Estrutura da tabela solicitacao_exclusao_historico

Coluna	Tipo	Observação
id, solicitacao_id	bigserial / bigint	histórico de mudanças de estado
estado_anterior, estado_novo	varchar(30)	de qual estado para qual
ocorrida_em, autor_id, observacao	timestampz / bigint / varchar	autor nulo quando é o usuário ou o sistema

Fonte: elaborado pelos autores (2026).

Quadro 28 – Estrutura da tabela notificacoes

Coluna	Tipo	Observação
id	bigserial	chave primária
usuario_id	bigint	obrigatório
imovel_id	bigint	opcional
titulo, mensagem, link	varchar	obrigatório (link opcional)
lida, criada_em	boolean / timestamp	obrigatório

Fonte: elaborado pelos autores (2026).

Além das colunas listadas, o esquema inclui a tabela de coleção `imovel_comodidades`, que armazena as comodidades de cada imóvel, e cinco índices: `idx_imoveis_usuario`, `idx_imoveis_ativo`, `idx_reservas_imovel`, `idx_reservas_periodo` (usado pela verificação de conflito de datas) e `idx_sugestoes_imovel_data`. A unicidade do endereço de e-mail do usuário é garantida por restrição de unicidade na própria coluna, não por índice nomeado. O fluxo de anúncios acrescentou os índices `idx_imoveis_status`,

idx_midias_imovel, idx_hist_preco_imovel, idx_aceites_imovel, idx_auditoria_imovel e idx_notificacoes_usuario, além da restrição de unicidade uk lembrete. As migrations V7 a V13 acrescentaram os campos comerciais e o snapshot da política, os pagamentos e reembolsos (com chaves de idempotência únicas), os bloqueios de datas, o SmartChat, o processamento de vídeo e o arquivamento seguido do descarte das colunas de WhatsApp. A migration V14 acrescentou a foto, a versão de sessão e o indicador de senha definida ao usuário, a auditoria de conta, os pedidos de troca de e-mail (com índice único parcial de um pedido pendente por usuário) e a solicitação de exclusão de dados com seu histórico (com índice único parcial de uma solicitação em andamento por usuário); nenhuma dessas tabelas guarda senha, hash em claro ou token, apenas o SHA-256 dos tokens de uso único. A migration V15 passou para TEXT as colunas de texto do SmartChat (mensagens, denúncias e a prévia das notificações), que agora guardam o valor cifrado no formato v1:Base64.

7 TELAS

Os protótipos de tela do SmartRent B2B já foram elaborados, mas ainda não haviam sido disponibilizados para incorporação a este relatório até o fechamento desta versão. O quadro a seguir relaciona as telas previstas para o MVP; será substituído pelas capturas reais assim que estas forem fornecidas.

Quadro 29 – Telas previstas para o MVP (pendente de anexo dos protótipos)

Tela	Propósito
Painel executivo	Visão consolidada de ocupação, receita prevista e evolução de preços sugeridos (RF10).
Gestão de imóveis	Listagem, cadastro e edição de imóveis (RF02 a RF04).
Gestão de reservas	Listagem, cadastro e cancelamento de reservas, com destaque para conflitos de data (RF05 a RF07, RF12).
Precificação inteligente	Solicitação e exibição da sugestão de preço, incluindo a justificativa da IA (RF08, RF09).
Meu perfil	Foto, nome de exibição, e-mail, senha e pedido de exclusão de dados, com o aviso das restrições temporárias (RF33 a RF39).

Fonte: elaborado pelos autores (2026).

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação – trabalhos acadêmicos – apresentação. Rio de Janeiro: ABNT, 2011.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6024**: informação e documentação – numeração progressiva das seções de um documento escrito – apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10520**: informação e documentação – citações em documentos – apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação – referências – elaboração. Rio de Janeiro: ABNT, 2018.

ORGANIZAÇÃO DAS NAÇÕES UNIDAS. **Objetivo de Desenvolvimento Sustentável 8**: trabalho decente e crescimento econômico. Nova York: ONU, 2015. Disponível em: <https://brasil.un.org/pt-br/sdgs/8>. Acesso em: 10 set. 2026.

SPRING. **Spring Boot Reference Documentation**. [S. l.]: Broadcom, [2024]. Disponível em: <https://docs.spring.io/spring-boot/>. Acesso em: 10 set. 2026.

HIBERNATE. **Hibernate ORM User Guide**. [S. l.]: Red Hat, [2024]. Disponível em: <https://hibernate.org/orm/documentation/>. Acesso em: 10 set. 2026.

GROQ. **Groq API Documentation**. [S. l.]: Groq, [2024]. Disponível em: <https://console.groq.com/docs>. Acesso em: 10 set. 2026.